تقرير تحقيقي موجز: تحديات PicoCTF

المحقق: [adoma32]

التحدي: 7 Forensics in PicoCTF

التاريخ: 27/11/2025

فئة التحقيق: Forensics (الإخفاء الرقمي - Steganography)

1\ Riddle Registry IN PicoCTF

Riddle Registry

Easy | Forensics | picoMini by CMU-Africa | browser_webshell_solvable

AUTHOR: PRINCE NIYONSHUTI N.

Description

Hi, intrepid investigator! 📄 🔍 You've stumbled upon a peculiar PDF filled with what seems like nothing more than garbled nonsense. But beware! Not everything is as it appears. Amidst the chaos lies a hidden treasure—an elusive flag waiting to be uncovered.
Find the PDF file here Hidden Confidential Document and uncover the flag within the metadata.

Hints ?

1 2

13,323 users solved

93% Liked

المقدمة والهدف:

يتناول هذا التقرير تفصيلاً لمنهجية حل تحدي Riddle Registry ضمن فئة Forensics (التحقيقات الرقمية) في مسابقة PicoCTF. يهدف التحدي إلى اختبار قدرة المحقق على استخلاص المعلومات المخفية ضمن البيانات الوصفية (Metadata) لملف رقمي، وهي تقنية شائعة في الإخفاء الرقمي (Steganography) والبحث الجنائي.

تحليل الملف الأولي (Initial File Analysis)

تم تقديم ملف للمستخدم، وغالباً ما يكون بصيغة وثيقة (Document Format)، مثل ملف PDF. كانت الخطوة الأولى في التحقيق هي التحقق من النوع الفعلي للملف باستخدام الأمر file للتأكد من سلامته وعدم التلاعب بامتداده:

Bash

```
file confidential.pdf
```

تبين أن الملف هو وثيقة PDF سليمة، مما أكد ضرورة التفتيش عن معلومات غير ظاهرة داخل هيكلها.

استخلاص البيانات الوصفية (Metadata Extraction)

تُعد البيانات الوصفية (Metadata) بمثابة سجل تفصيلي يحتوي على معلومات عن مُنشئ الملف وتاريخ إنشائه وأي تعليقات أو علامات (Tags) أُضيفت إليه. ولأجل استخراج هذه البيانات، تم استخدام الأداة المتخصصة exiftool، وهي أداة قوية لتحليل بيانات الميديا والوثائق:

Bash

```
exiftool confidential.pdf
```

النتائج والملاحظات:

1. باستعراض مخرجات (Output) الأمر exiftool، تم التركيز على حقول البيانات التي تسمح بإدخال نصوص مخصصة، مثل حقل "Author" (المؤلف) أو "Creator" (المنشئ).
2. لوحظ أن أحد هذه الحقول لا يحتوي على اسم تقليدي، بل على سلسلة طويلة من الأحرف والأرقام والرموز، وينتهي عادةً بعلامة التساوي (=)، مما يشير بقوة إلى أن النص قد خضع لعملية تشفير (Encoding).

مرحلة فك التشفير (Decoding)

تبين أن السلسلة النصية المشبوهة تتبع النمط القياسي لترميز Base64، وهو نوع من الترميز يستخدم لتحويل البيانات الثنائية (Binary Data) إلى نص يمكن نقله بأمان عبر البروتوكولات النصية.

لإتمام عملية فك التشفير، تم استخدام إحدى طريقتين:

1. عبر طرفية (Terminal) Linux: باستخدام الأمر echo لتمرير السلسلة المشفرة إلى أداة فك تشفير base64 --decode:

Bash

```
echo 'السلسلة_المشفرة_المستخلصة_من_exiftool' | base64 --decode
```

2. عبر أداة خارجية (Online Tool): باستخدام اداة CyberChef لإدخال السلسلة وفك ترميزها.

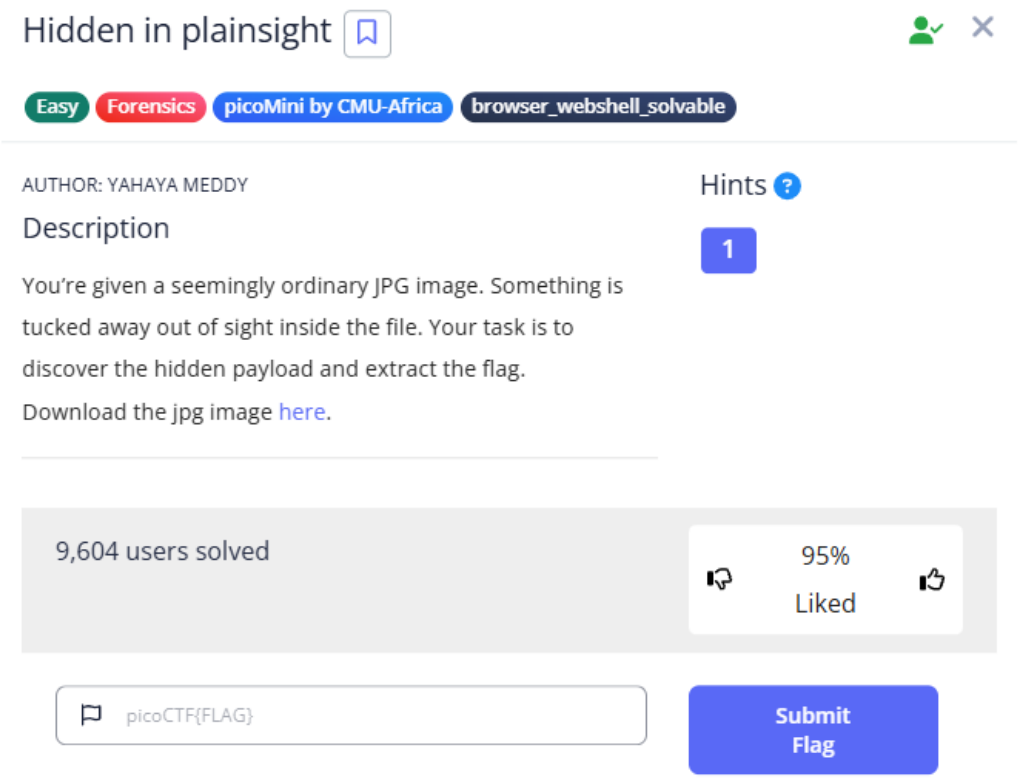
النتيجة النهائية (The Flag)

بعد تطبيق عملية فك تشفير Base64 على السلسلة المستخرجة، ظهرت قيمة النص الأصلي، وكانت تطابق الشكل القياسي لـ Flag الخاص بمسابقة PicoCTF، مؤكداً بذلك نجاح عملية التحقيق:

الحقل المستهدف	القيمة المشفرة (Base64)	القيمة بعد فك التشفير (The Flag)
Author (مثال)	=cGljb0NURntwdXp6bDNkX20zdGFkYXRhX2YwdW5kIV9jOGY5MWQ2OH0	picoCTF{puzzl3d_m3tadata_f0und!_c8f91d68}

بهذا، تم حل التحدي من خلال تطبيق مبدأ البحث في البيانات الوصفية للملفات باستخدام الأداة المناسبة.

2\Hidden in plainsight



1. الإجراءات المتبعة (Actions Taken)

بصفتي محققاً، ركزت الإجراءات على استخلاص البيانات المخفية ضمن ملف الصورة المُقدم (img.jpg). لقد قمت تنفيذ الآتي:

- تحليل البيانات الوصفية (Metadata Analysis): قمت باستخدام الأداة exiftool لفحص جميع حقول الـ Metadata للصورة، بحثاً عن أي تلميحات غير قياسية أو سلاسل مشفرة.
- فك الترميز الأولي (Initial Decoding): وجدت سلسلة مشفرة بنظام Base64 في حقل الـ Comment . ثم قمت بفك ترميز هذه السلسلة باستخدام الأمر base64 --decode.
- تحديد مفتاح الإخفاء: كشفت عملية فك الترميز عن تلميح واضح يحدد كلمة المرور (Passphrase) وأداة الإخفاء المستخدمة، وهي Steghide.
- استخراج البيانات المخفية (Steganographic Extraction): قمت باستخدام الأداة steghide مع كلمة المرور المستخلصة لاستخراج ملف البيانات المخفي من داخل الصورة.

2. النتائج المُتحققة (Findings)

قادت الإجراءات المذكورة أعلاه إلى النتائج التالية:

- اكتشاف ملف flag.txt: نجحت عملية الـ Steganography في استخراج ملف نصي باسم flag.txt.
- استرداد العلم (Flag Recovery): احتوى الملف المستخرج على الـ Flag المطلوب لإنهاء التحدي.

3. الخلاصة (Conclusion)

تم حل التحدي بنجاح من خلال ربط تقنية تحليل Metadata كوسيلة للحصول على المفتاح اللازم لفك إخفاء Steganography. يتأكد من هذا التحدي أن البيانات الوصفية غالباً ما تُستخدم كـ Registry لمفاتيح فك الإخفاء.

العلم النهائي المستخلص:

picoCTF{h1dd3n_1n_1m4g3_871ba555}

3\Flag in Flame

Flag in Flame

Easy | Forensics | picoMini by CMU-Africa | browser_webshell_solvable

AUTHOR: PRINCE NIYONSHUTI N.

Description

The SOC team discovered a suspiciously large log file after a recent breach. When they opened it, they found an enormous block of encoded text instead of typical logs. Could there be something hidden within? Your mission is to inspect the resulting file and reveal the real purpose of it. The team is relying on your skills to uncover any concealed information within this unusual log.
Download the encoded data here: Logs Data. Be prepared—the file is large, and examining it thoroughly is crucial .

Hints

1

. الإجراءات المتبعة (Actions Taken)

تضمن التحدي التعامل مع ملف سجل نظام (Log File) ضخم ومُحمّل ببيانات مُرمّزة، مما استلزم تصفية البيانات وفك ترميزها على مراحل. تم تنفيذ الآتي:

- عزل كتلة البيانات: تم تحديد كتلة بيانات كبيرة جداً داخل سجل النظام، حيث ظهرت كـ Symmetric Block من الترميز الأولي (غالباً Base64).
- فك الترميز المتعدد (Multi-level Decoding):
  - المستوى الأول: تم فك ترميز الكتلة الكبيرة من Base64 باستخدام الأمر base64 --decode.
  - النتيجة: لم ينتج نص مقروء (ASCII Text)، بل تم استخراج بيانات ثنائية (Binary Data).
  - الامر : cat logs.txt | base64 --decode > out

- تحديد نوع الملف (File Identification): تم استخدام الأمر file على البيانات الثنائية المستخرجة لتحديد هويتها الحقيقية.
- التحويل والاستخراج: كشفت عملية الـ Identification أن البيانات الثنائية هي في الواقع صورة (PNG image data). تم تغيير امتداد الملف (File Extension) لفتحه.

2. النتائج المُتحققة (Findings)

قادت العملية إلى استرداد صورة، لكنها لم تكن تحمل الـ Flag كـ Text عادي، بل كان مُرمّزاً مرة أخرى داخل الصورة:

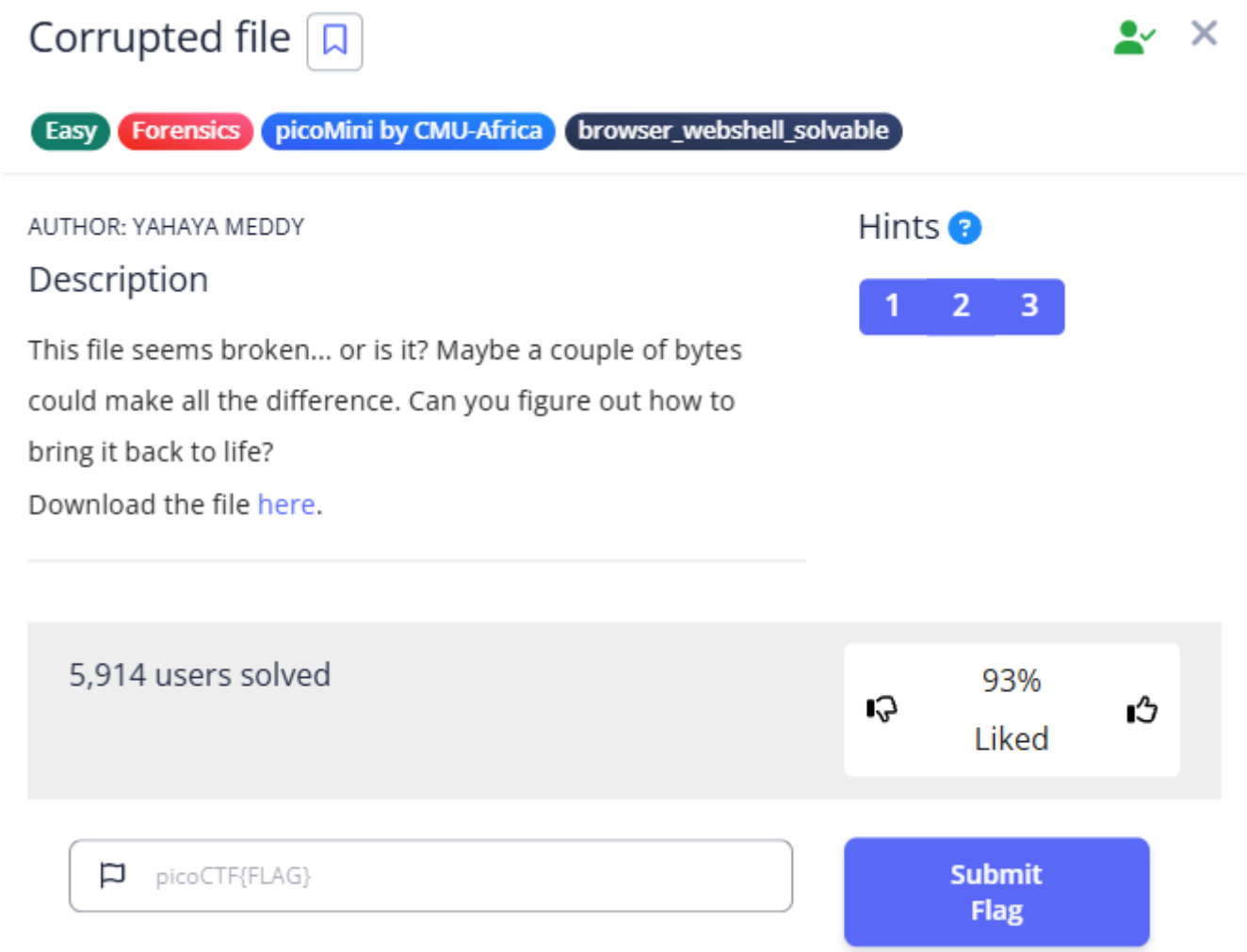
- اكتشاف ترميز Hex: تم العثور على سلسلة طويلة من الأحرف والأرقام تُمثل ترميز Hexadecimal (سداسي عشري).
- فك ترميز Hex: تم استخدام الأمر xxd -p -r لفك ترميز سلسلة Hex إلى نص ASCII مقروء.

3. الخلاصة (Conclusion)

تم حل التحدي من خلال تطبيق منهجية الطبقات في التحقيق، حيث قاد فك الترميز من Base64 إلى صورة، وقادت الصورة إلى مفتاح آخر على شكل Hex Encoding.

العلم النهائي المستخلص: picoCTF{forensics_analysis_is_amazing_2561a194}

4\Corrupted file



1. الإجراءات المتبعة (Actions Taken)

تم التعامل مع ملف صورة تالف (file) لا يفتح بسبب تلف جزئي في التوقيع السحري (Magic Bytes) الخاص به. تم تنفيذ الإجراءات التالية:

- تحليل الـ Header التالف: تم فحص البايتات الافتتاحية للملف باستخدام Hex Editor، والتي كشفت أن الملف كان مخصصاً ليكون بصيغة JPEG (لظهور توقيع FF D8)، ولكنه لم يكن سليماً بالكامل.
- تصحيح الـ Header يدوياً (Manual Repair): تم استخدام أدوات الـ Command Line لتكوين ملف جديد عن طريق إزالة الجزء التالف وإعادة بناء الـ Magic Bytes الصحيحة:

```
(printf '\xff\xd8' && tail -c +3 file) > out.jpg
```

- الأمر printf '\xff\xd8' طبع أول بايتين صحيحين لتوقيع JPEG.
- الأمر tail -c +3 file قام بقص وتجاهل أول بايتين من الملف التالف، وبدأ أخذ البيانات من البايت الثالث فصاعداً.
- تم دمج النتيجتين وتوجيههما إلى ملف سليم جديد اسمه out.jpg.

- استرداد العلم (Flag Recovery): تم فتح الملف المُصلح (out.jpg)، حيث ظهرت الصورة بوضوح وتم استخراج الـ Flag منها.

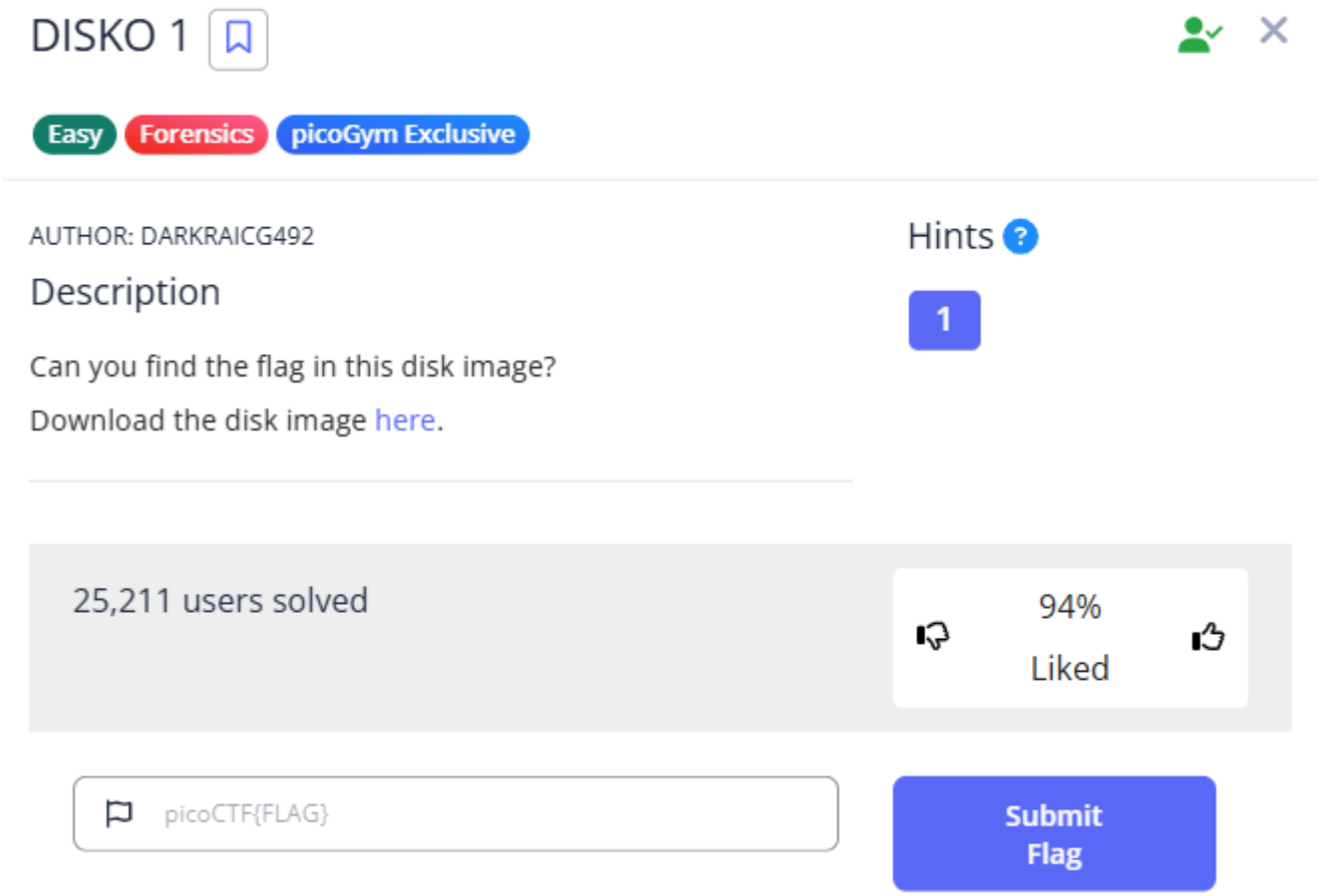
2. الخلاصة (Conclusion)

تم حل التحدي بنجاح عن طريق تطبيق تقنية إصلاح ترويسة الملف (Header Repair) يدوياً. أثبت هذا التحدي أن التحقق من الـ Magic Bytes وتصحيحها هو خطوة أساسية في التعامل مع الملفات التالفة في الـ Forensics.

العلم النهائي المستخلص:

picoCTF{r3st0r1ng_th3_by73s_752d2c00}

5\DISKO 1



1. الإجراءات المتبعة (Actions Taken)

تم التعامل مع ملف صورة قرص مضغوط (Compressed Disk Image)، مما استلزم إجراء تحليلات أساسية للبيانات الخام (Raw Data) بدلاً من تحليل نظام الملفات (File System) المعقد. تم تنفيذ الإجراءات الآتية:

- فك الضغط الأولي (Initial Decompression): تم تحديد أن الملف هو صورة قرص مضغوطة باستخدام أداة gzip (أو أمر مماثل). تم فك ضغط الملف لاستخراج البيانات الخام (Raw Data):

```
gzip -d disk.img.gz
```

- استخراج النصوص (String Extraction): نظراً لاحتمالية أن يكون العلم (Flag) مُخزناً كنص عادي (Plain Text) داخل البيانات الخام، تم استخدام الأداة strings لاستخراج كل السلاسل النصية القابلة للقراءة من ملف الـ Image.

- تصفية النتائج (Filtering and Grep): تم تصفية المخرجات النصية الناتجة للبحث عن التوقيع القياسي لأعلام PicoCTF باستخدام الأمر grep:

```
Strings disko-1.dd | grep picoCTF
```

2. النتائج المُتحققة (Findings)

كشفت عملية التصفية المباشرة عن سلسلة نصية واحدة تطابق شكل العلم، مما يشير إلى أن الـ Flag لم يكن مخفياً داخل ملفات System Structure وإنما كان موجوداً كـ String عادي ضمن بيانات الـ Disk Image الخام.

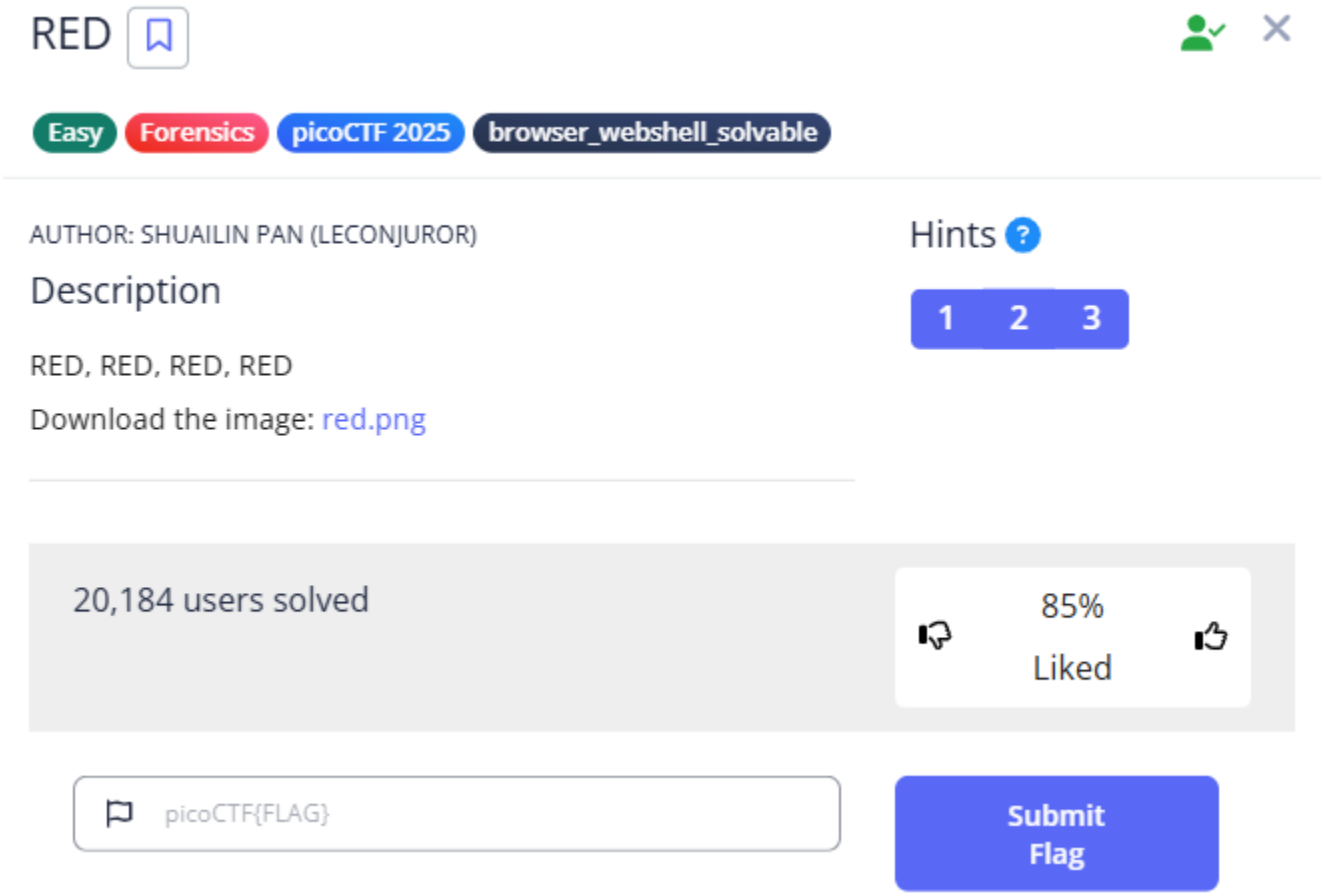
العلم النهائي المستخلص:

picoCTF{1t5_ju5t_4_5tr1n9_be6031da}

3. الخلاصة (Conclusion)

تم حل التحدي بنجاح باستخدام منهجية بسيطة تعتمد على التصفية المباشرة للبيانات الخام بعد فك ضغطها. يؤكد هذا التحدي أهمية البدء بأبسط أدوات التحقيق (مثل strings و grep) قبل اللجوء إلى تقنيات تركيب نظام الملفات (Mounting File Systems) الأكثر تعقيداً.

6\RED (Steganalysis)



1. الإجراءات المتبعة (Actions Taken)

تم التعامل مع ملف صورة من نوع PNG، والمطلوب كان الكشف عن بيانات مخفية فيه، مما قاد لاستخدام تقنيات الـ Steganalysis. تم تنفيذ الإجراءات التالية:

- تحليل الإخفاء الرقمي (Steganalysis): بدلاً من الفحص اليدوي للـ Metadata، تم استخدام الأداة المتخصصة zsteg لفحص الصورة red.png. تُستخدم هذه الأداة لاكتشاف البيانات المخفية في أقل البتات أهمية (LSB - Least Significant Bit):

Bash

```
zsteg red.png
```

- اكتشاف الترميز (Encoding Discovery): أسفرت مخرجات الأمر zsteg عن اكتشاف سلسلة نصية طويلة غير مقروءة، وتم تحديدها على الفور كترميز Base64 نظراً لشكلها وأحرفها المميزة.

- فك الترميز (Base64 Decoding): تم استخراج سلسلة Base64 المكتشفة وتم فك ترميزها باستخدام الأمر القياسي Base64

2. النتائج المُتحققة (Findings)

قادت عملية فك الترميز مباشرةً إلى العلم (Flag) المطلوب.

العلم النهائي المستخلص:

picoCTF{r3d_1s_th3_ult1m4t3_cur3_f0r_54dn355_}

3. الخلاصة (Conclusion)

تم حل التحدي بنجاح من خلال تطبيق أدوات متخصصة في تحليل الإخفاء الرقمي (مثل zsteg) لتحديد مكان البيانات المخفية، تلاها خطوة بسيطة لفك الترميز (Base64 Decoding). يؤكد هذا التحدي أهمية استخدام الأداة المناسبة لاكتشاف تقنية الإخفاء المستخدمة.

7\Ph4nt0m 1ntrud3

Ph4nt0m 1ntrud3r

Easy | Forensics | picoCTF 2025 | browser_webshell_solvable

AUTHOR: PRINCE NIYONSHUTI N.

Description

A digital ghost has breached my defenses, and my sensitive data has been stolen! 😱💻 Your mission is to uncover how this phantom intruder infiltrated my system and retrieve the hidden flag.
To solve this challenge, you'll need to analyze the provided PCAP file and track down the attack method. The attacker has cleverly concealed his moves in well timely manner. Dive into the network traffic, apply the right filters and show off your forensic prowess and unmask the digital intruder!
Find the PCAP file here Network Traffic PCAP file and try to get the flag.

Hints

1 2 3

r

1. ملخص المهمة (Mission Abstract)

الهدف هو تحليل ملف Packet Capture (myNetworkTraffic.pcap) لتتبع نشاط متسلل واستخراج العلم (Flag) المخفي. تطلب التحدي إعادة تجميع بيانات مجزأة ومخفية داخل حزم الشبكة.

2. العقبات والتحول في المنهجية (Pivot & Obstacles)

- المحاولة الأولية (GUI Approach): بدأت التحقيق باستخدام واجهة Wireshark الرسومية. ورغم كفاءتها في التصفح العام، تبين أن استخراج البيانات المطلوبة يدوياً عملية مستهلكة للوقت وغير عملية (Time-Consuming)، نظراً لتجزئة الـ Flag إلى قطع صغيرة جداً موزعة على حزم متعددة.
- الحل البديل (CLI Approach): قررت التحول إلى استخدام أداة سطر الأوامر tshark لأتمتة عملية الاستخراج والفلترة، مما سمح بمعالجة آلاف الحزم في ثانية واحدة.

3. التنفيذ التقني (Technical Execution)

تم صياغة أمر مركب (One-Liner) يقوم بعدة عمليات جنائية متسلسلة لاستخراج العلم:

Bash

```
tshark -r myNetworkTraffic.pcap -Y "tcp.len==12 || tcp.len==4" -T fields -e frame.time -e tcp.segment_data | sort -k4 | awk '{print $6}' | xxd -p -r | base64 -d
```

تحليل الأمر (Command Breakdown):

1. استخراج البيانات الموجه (Targeted Extraction):
   - tshark -r ...: قراءة الملف.
   - -Y "tcp.len==12 || tcp.len==4": هذا هو الفلتر الذكي. تم ملاحظة أن الحزم التي تحتوي على أجزاء الاعلام لها طول بيانات محدد (إما 12 أو 4 بايتات)، مما عزل حزم المتسلل عن باقي ضجيج الشبكة.

2. تحديد الحقول (Field Selection):
   - -T fields -e frame.time -e tcp.segment_data: استخراج وقت الحزمة (للترتيب) والبيانات المحمولة (Payload) فقط.
3. إعادة الترتيب (Reassembly):
   - sort -k4: ترتيب الحزم زمنياً لضمان تجميع أحرف العلم بالترتيب الصحيح.
4. التنظيف وفك التشفير (Decoding Pipeline):
   - awk '{print $6}': عزل عمود البيانات (Data Column) فقط.
   - xxd -p -r: تحويل البيانات من صيغة Hex Dump إلى بيانات خام (Binary/ASCII).
   - base64 -d: فك تشفير البيانات المجمعة (التي كانت مرمزة بـ Base64) للكشف عن النص الأصلي.

4. النتائج (Findings)

نجح الأمر في تجميع الأحرف المتناثرة وفك تشفيرها فوراً، مما أظهر العلم بوضوح دون الحاجة لتتبع عشرات الـ TCP Streams يدوياً.

العلم النهائي المستخلص:

picoCTF{1t_w4snt_th4t_34sy_tbh_4r_8e10e839}

5. الخلاصة (Conclusion)

أثبت هذا التحدي أن الاعتماد الكلي على الواجهات الرسومية (GUI) قد يكون عائقاً في تحديات الـ Data Reassembly المعقدة. استخدام Scripting و tshark وفر دقة أعلى واختصر وقتاً طويلاً، وهو الأسلوب الأمثل للتعامل مع الـ Fragmentation المتعمد من قبل المهاجمين.